

Aprendizado por Reforço em Ambiente Grid-World para Navegação e Evasão de Obstáculos

1st Brithany Michelle Oliva Chuquimia

Matricula 971

Inatel

Santa Rita do Sapucaí-MG-Brasil

2nd Luis Enrique Cardozo Ramirez

Matricula 949

Inatel

Santa Rita do Sapucaí-MG-Brasil

Abstract—Este trabalho apresenta o desenvolvimento e a avaliação de um agente de navegação baseado no algoritmo *Advantage Actor-Critic* com memória LSTM (A2C-LSTM) aplicado a um ambiente *GridWorld* estático e dinâmico. O ambiente modelado inclui obstáculos estáticos e dinâmicos, limitação perceptual (POMDP) e diferentes esquemas de recompensa, incluindo técnicas de *Potential-Based Reward Shaping* (PBRs). Foram investigadas 12 combinações entre funções de recompensa e estratégias de treinamento, com a configuração orientada à rapidez apresentando o melhor desempenho. Os resultados demonstram que o agente alcança convergência rápida, capacidade de adaptação frente a bloqueios inesperados. Os achados evidenciam a importância da memória temporal e de recompensas densas para garantir navegação robusta e eficiente.

Index Terms—Aprendizado por Reforço, Navegação de Robôs, Grid-world, Inteligência Artificial, Sistemas Autônomos.

I. INTRODUÇÃO

O Aprendizado por Reforço (RL) é um dos pilares da IA, no qual um agente aprende decisões sequenciais interagindo com um ambiente dinâmico. Diferentemente do aprendizado supervisionado, o RL não utiliza rótulos explícitos; o comportamento do agente é guiado por recompensas que avaliam suas ações, promovendo aprendizado por tentativa e erro [1]. Assim, o agente desenvolve políticas que maximizam a recompensa acumulada, mesmo em cenários incertos ou parcialmente observáveis.

Tarefas de navegação e evasão de obstáculos são aplicações clássicas de RL, presentes em robótica móvel, veículos autônomos e agentes virtuais. Nesses contextos, o agente deve planejar trajetórias seguras e eficientes em ambientes com restrições e elementos dinâmicos. Para estudar tais propriedades de forma controlada, utiliza-se frequentemente o ambiente *grid-world*, uma abstração bidimensional que permite analisar a convergência, a exploração e os efeitos do esquema de recompensas. Sua simplicidade computacional torna-o um laboratório ideal antes de aplicações mais complexas.

Neste trabalho, investigou-se a aprendizagem de políticas de navegação em um ambiente bidimensional com obstáculos, avaliando a capacidade do agente em atingir metas de forma segura e com trajetórias curtas. A análise considerou métricas como taxa de sucesso, número médio de passos e estabilidade da política durante o treinamento. O código, modelos e materiais complementares estão disponíveis em: <https://github.com/luisenriqueCR/TP557/tree/main/ProjetoFinal>.

II. TRABALHOS RELACIONADOS

Este trabalho apoia-se em fundamentos clássicos de Aprendizado por Reforço (RL), onde problemas de navegação em *grid-world* são amplamente utilizados para estudar a convergência de políticas e o impacto das funções de recompensa em ambientes discretos [1]. A adoção do método Actor-Critic justifica-se por sua capacidade de estimar simultaneamente política e valor, oferecendo estabilidade e eficiência em comparação com abordagens puramente baseadas em valor ou política [2]. Além disso, estudos recentes em tomada de decisão sequencial destacam a relevância da otimização de trajetórias e da evasão de colisões como desafios centrais em sistemas autônomos [6], alinhando-se diretamente aos objetivos do presente trabalho.

III. METODOLOGIA

O aprendizado foi realizado com algoritmos de RL, usando redes neurais para "aproximar" a função que decide qual ação o agente deve tomar. A abordagem preferencial foi Actor-Critic, pois permitiu aprender simultaneamente uma política de ações (actor) e uma função de valor (critic), sendo adequada para problemas contínuos ou de alta dimensionalidade.

1) **Implementação do ambiente:** O ambiente consiste em um *GridWorld* bidimensional, este cenário foi projetado para simular tarefas de navegação e evasão de obstáculos, incorporando tanto elementos estáticos quanto dinâmicos. A representação do ambiente, estruturada em uma matriz 5×5 e ilustrada na Fig. 1, define-se cada célula do grid como um tipo específico de espaço:

- **Espaço livre (0)** – área navegável onde o agente pode se deslocar;
- **Obstáculo (-1)** – célula bloqueada, cuja colisão resulta em penalização;
- **Meta (2)** – posição objetivo que encerra o episódio com recompensa positiva.

A implementação contempla dois modos de observação, inspirados em diferentes níveis de percepção do agente:

- **Modo MDP (Markov Decision Process)** – fornece ao agente observabilidade completa do ambiente, representada por quatro canais de entrada: obstáculos, objetivo, posição atual e histórico de visitas;

- **Modo POMDP (Partially Observable MDP)** – limita a percepção do agente a uma região local do mapa, complementada por uma codificação *one-hot* da sua posição. Esse modo simula condições mais realistas, em que a visão do agente é restrita.

Os obstáculos foram definidos em dois tipos principais:

- **Estáticos:** Permanecem fixos durante o episódio;
- **Dinâmicos:** Deslocam-se a cada passo do tempo, de forma aleatória (*random*) ou segundo padrões pré-definidos (*pattern*).

O ambiente foi desenvolvido em Python no Google Colab, de forma parametrizável e modular, permitindo ajustar dimensões, obstáculos e recompensas de acordo com cada experimento.

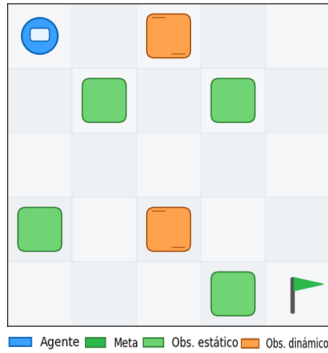


Fig. 1. Ambiente GridWorld 5x5

2) **Função de recompensas e penalizações:** Foi projetada para equilibrar o aprendizado do agente entre eficiência e segurança, incentivando ações que resultam em progresso e desestimulando comportamentos ineficientes ou inseguros. Diversas configurações foram testadas, para avaliar o impacto de cada parâmetro sobre a estabilidade e a velocidade de convergência do aprendizado.

a) **Configuração da Função de Recompensa** Foram definidas quatro versões diferentes da função de recompensa, cada uma representando um tipo de comportamento esperado no agente:

- Equilibrada:** Equilíbrio entre custo por passo e recompensa final.
- Rapidez:** Favorece trajetos curtos com alto risco inicial; prioriza resultados rápidos.
- Exploração:** Penalizações suaves que incentivam o agente a explorar sem medo de errar.
- Segurança:** Altas penalizações para erros; o agente evita riscos, mas torna-se mais lento.

Após a análise comparativa, a configuração de rapidez foi selecionada como base para os experimentos finais, por apresentar maior estabilidade em cenários com obstáculos dinâmicos e observação parcial.

b) **Configuração de Rapidez e Reward Shaping (PBRS)** A função de recompensa influencia na es-

tabilidade, velocidade de convergência e a propensão do agente a explorar comportamentos seguros ou arriscados. Foram avaliadas 2 categorias de configuração: (i) um esquema orientado à rapidez, e (ii) variantes com *Potential-Based Reward Shaping* (PBRS).

• **Configuração de rapidez:** A configuração rapidez utiliza penalizações fortes por passo e uma recompensa final elevada, incentivando o agente a minimizar o tempo até o objetivo. Ela atua como mecanismo de estabilização em ambientes com obstáculos dinâmicos, evitando movimentos improdutivos, colisões e revisitas. Os parâmetros empregados estão resumidos na Tabela I.

TABLE I
PARÂMETROS DA CONFIGURAÇÃO DE RECOMPENSA VOLTADA À SEGURANÇA

Evento / Estado	Recompensa
Objetivo alcançado (goal)	+50.0
Colisão com parede (wall)	-10.0
Sair dos limites	-10.0
Passo realizado (step)	-1.0
Revisitar célula	-0.05
Sem movimento Ação inválida (no_move)	-1.0

• **Configurações com Reward Shaping (PBRS):**

Para acelerar o aprendizado inicial, foram introduzidos termos adicionais de recompensa baseados em *Potential-Based Reward Shaping* (PBRS), técnica formalmente provada para preservar a política ótima [7]. O PBRS atua como "nível energético" que incentiva movimentos que reduzem a distância ao objetivo, sem alterar a política ótima teórica. Isto garante:

- maior densidade de sinal de recompensa.
- convergência mais rápida.
- redução de exploração aleatória.

O esquema PBRS adiciona uma heurística estruturada ao aprendizado, guiando o agente em direção ao objetivo enquanto mantém as garantias formais da estrutura de vantagem do A2C.

3) **Arquitetura da Rede Neural A2C com LSTM:**

A arquitetura proposta baseia-se no algoritmo *Advantage Actor-Critic* (A2C) combinado com memória temporal por meio de redes *Long Short-Term Memory* (LSTM), visando a navegação em ambientes com obstáculos dinâmicos.

a) **Representação do Estado** Foram consideradas duas formas distintas de representar o estado do ambiente, correspondentes aos modos MDP e POMDP. Ambos foram avaliados em experimentos preliminares, com o modo POMDP sendo adotado na versão final dos testes dinâmicos.

• **Modo MDP (observabilidade total):** Neste modo, o agente tem acesso completo ao grid em

cada passo. O estado é composto por a máscara de obstáculos, a máscara do objetivo, a posição do agente codificada em *one-hot encoding*, e um mapa de visitas (histórico). Essa representação é empacotada em um tensor $5 \times 5 \times 4$, posteriormente achatado para 100 dimensões.

- **Modo POMDP (observação parcial):** O agente não tem acesso completo ao ambiente. Nesse caso, o estado é composto por: (i) mapa interno construído pelo agente, (ii) Posição atual em one-hot (5×5) O vetor final possui 50 dimensões, resultante do achatamento e concatenação dessas matrizes 5×5 . A diferença fundamental é que, no POMDP, o agente precisa usar memória temporal para inferir informações não observadas diretamente, justificando o uso da camada LSTM na arquitetura.

b) **Arquitetura Actor-Critic e memória LSTM**

A rede adota uma arquitetura *dual*, composta por dois ramos paralelos que recebem o mesmo estado de entrada. O modelo A2C (Advantage Actor-Critic) baseia-se em uma arquitetura de aprendizado, as duas redes neurais operam de forma cooperativa.

- **Ator (policy network):** responsável por determinar a probabilidade de cada ação possível a partir do estado corrente, definindo a política estocástica $\pi(a|s)$. A Rede Actor foi composta por uma camada LSTM com 128 unidades ocultas, seguida de uma camada totalmente conectada que mapeia para 4 ações (cima, baixo, esquerda e direita). A saída é normalizada por *softmax* e tratada como uma distribuição categórica para seleção estocástica de ações.
- **Crítico (value network):** encarregado de estimar o valor escalar $V(s)$ de cada estado, o que permite calcular a *vantagem*:

$$A(s, a) = R_t + \gamma V(s') - V(s)$$
 Esse valor serve como um sinal de correção para orientar o ator, reduzindo a variância e promovendo estabilidade nas atualizações. A Rede Critic utiliza a mesma estrutura LSTM com 128 unidades, seguida de uma camada linear responsável por estimar o valor escalar $V(s)$ do estado.

A Tabela II apresenta a arquitetura empregada no agente A2C-LSTM, bem como os parâmetros do ambiente dinâmico utilizado nos experimentos.

4) **Treinamento do agente:**

O treinamento do agente segue o esquema clássico do algoritmo Advantage Actor-Critic (A2C), no qual os parâmetros do ator e do crítico são atualizados com base na vantagem estimada e em um mecanismo de estabilização que inclui GAE, entropia e action masking. A seguir, descrevem-se os principais componentes desse processo.

TABLE II
ARQUITETURA DO AGENTE A2C-LSTM E DINÂMICA DO AMBIENTE.

Componente	Valor
Modo de estado	POMDP
Dimensão do estado	50
Hidden LSTM	128
Ações	4
Ator	LSTM→FC(4)
Crítico	LSTM→FC(1)
Estados ocultos	(h_0, c_0) Reset por episódio
Obstáculos (modo)	Aleatório
Prob. permanecer	0.50
Atualização	Cada passo
Ordem	Obst.→Agente
Sementes	3–10

- a) **Função de perda do A2C** Os pesos das redes Actor e Critic são atualizados a cada passo utilizando três termos fundamentais:

- **Policy loss:** Incentiva ações com vantagem positiva.
- **Value loss:** Penaliza erros na estimativa de valor.
- **Entropy loss:** Promove exploração adicionando aleatoriedade controlada à política:

A perda total utilizada é:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + \alpha \mathcal{L}_{\text{value}} - \beta \mathcal{H}(\pi)$$

onde $\alpha = 0.5$ e β é ajustado dinamicamente.

- b) **Exploração e Entropia em A2C:** No experimento final, utilizou-se um coeficiente de entropia constante $\beta = 0.02$, o que mantém um nível estável de aleatoriedade na política durante todo o treinamento. Esse valor mostrou-se suficiente para:

- evitar convergência prematura
- manter diversidade de trajetórias
- permitir adaptação ao ambiente dinâmico
- favorecer convergência rápida da política

- c) **Variações no Processo de Treinamento** Foram testadas três abordagens principais de treinamento, que diferem no grau de entropia inicial e intensidade de regularização. Todas utilizam o mesmo arcabouço *Actor-Critic* recorrente (A2C-LSTM) com *action masking*, *reward shaping* (PBRS) e vantagem generalizada (GAE).

- Exploratório:** Maior coeficiente de entropia; favorece descoberta de trajetórias alternativas.
- Voltado à rapidez:** Entropia reduzida desde o início; favorece políticas mais determinísticas.
- Equilibrado:** Níveis intermediários de entropia, balanceando exploração e estabilidade.

- d) **Avaliação combinada das configurações.**

Para selecionar a configuração mais adequada ao ambiente com obstáculos dinâmicos, foram avaliadas 12 combinações resultantes da interseção entre: as quatro variações da função de recompensa, e as três estratégias de treinamento.

Cada combinação foi avaliada em sessões preliminares, analisando-se estabilidade da política, taxa de sucesso, colisões e comprimento médio das trajetórias. Os resultados globais dessas avaliações são apresentados na Fig. 2, que ilustra a comparação das recompensas normalizadas e o trade-off entre sucesso e eficiência.

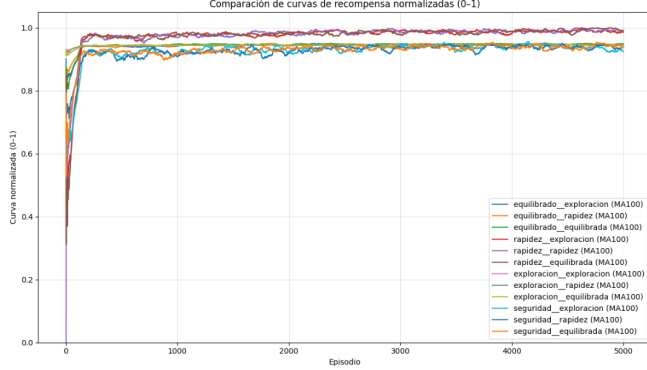


Fig. 2. Ambiente GridWorld 5x5

Com base nesses testes comparativos, constatou-se que a combinação de:

- recompensa do tipo rapidez
- treinamento igualmente voltado à rapidez

o qual apresentou o melhor desempenho composto entre todas as 12 combinações avaliadas.

- e) **Parâmetros de treinamento.** Para garantir a consistência entre as execuções, adotaram-se uma configuração com parâmetros ajustados para recompensar trajetórias eficientes e acelerar o aprendizado Conforme apresentado na Tabela III, os principais hiperparâmetros do treinamento incluem:

TABLE III
HIPERPARÂMETROS DE TREINAMENTO E OTIMIZAÇÃO.

Parâmetro	Valor
Episódios	5 000
Máx. passos por episódio	300
γ (discount factor)	0.99
GAE (λ)	0.95
Coefficiente de entropia	0.02
Coefficiente de valor (value_coef)	0.6
Uso de GAE	ativado
Frequência de logs (print_every)	200
Semente (seed)	2
Otimizadores	
Algoritmo	Adam
Taxa de aprendizado (ator)	3×10^{-4}
Taxa de aprendizado (crítico)	8×10^{-4}
Parametro eps (critic Adam)	1×10^{-5}

IV. RESULTADOS E AVALIAÇÃO EXPERIMENTAL

Esta seção apresenta os resultados obtidos no treinamento do agente A2C-LSTM no ambiente GridWorld dinâmico. As

métricas analisadas incluem a evolução das recompensas, taxa de sucesso, comprimento dos episódios e avaliação determinística pós-treinamento. Os resultados refletem o comportamento do agente sob o esquema de recompensas *rapidez* e o perfil de treinamento *rapidez*, previamente identificada como a combinação de melhor desempenho entre as 12 variações avaliadas..

A. Evolução das Recompensas

A Fig. 3 mostra a evolução da recompensa acumulada e da média móvel (MA100) durante os 5000 episódios de treinamento. Nos primeiros episódios, o agente apresenta recompensas negativas devido à forte penalização por passo da função de recompensa orientada à rapidez. Entretanto, a convergência ocorre rapidamente: por volta dos 200~300 episódios, a MA100 torna-se positiva e continua crescendo de forma consistente.

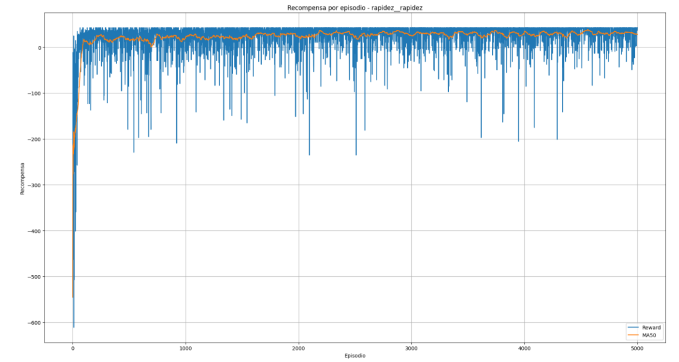


Fig. 3. Ambiente GridWorld 5x5

Entre os episódios 1000 e 5000, observam-se janelas inteiras com MA100 superior a +25, refletindo trajetórias curtas, decisões agressivas e elevada eficiência temporal — características esperadas da função de recompensa rapidez.

Esses resultados demonstram que o agente aprendeu rapidamente uma política eficaz para minimizar o número de passos e maximizar a recompensa total.

B. Taxa de Sucesso

A taxa de sucesso manteve-se em **100%** ao longo de todo o treinamento. Mesmo com variabilidade natural causada pelo comportamento agressivo induzido pela função de rapidez, o agente conseguiu alcançar o objetivo de maneira consistente em praticamente todos os episódios. A combinação entre a arquitetura recorrente (LSTM) e a política estocástica estabilizada pelo termo de entropia constante ($\beta = 0.02$) contribuiu para manter a capacidade de adaptação às mudanças do ambiente dinâmico.

C. Avaliações Específicas: Teste A, B e C

Além da avaliação determinística, foram realizados três testes estruturados para analisar a capacidade de adaptação, desvio e robustez do agente.

1) Teste A — Desempenho em ambiente estático

Objetivo: avaliar a política em um ambiente sem obstáculos móveis.

Resultados:

- sucesso: 100%
- comprimento: 8 passos,
- nenhuma penalização significativa.

Esse teste estabelece a rota de referência (“rota base”) utilizada nas análises seguintes.

- 2) **Teste B — Bloqueio da rota base e reencaminhamento** Objetivo: avaliar capacidade de desvio quando um obstáculo é inserido artificialmente na rota ótima. Resultados:

- trajeto base: 9 passos,
- trajeto com bloqueio: 11 passos,
- desvio adicional: 2 passos,
- divergência no passo 4,
- reencaminhamento no passo 4,
- colisões ocasionais durante o processo (2 colisões com parede e revisitas)

O agente adapta-se imediatamente ao distúrbio e encontra um caminho alternativo, demonstrando comportamento reativo e planejamento dinâmico.

- 3) **Teste C: Teste de resistência a obstáculos dinâmicos** Objetivo: medir robustez estatística frente a múltiplas execuções independentes com dinâmica aleatória.

Resultados agregados:

- success_rate = 100%,
- ep_len médio = 12.1,
- ep_len mediano = 11,
- colisões totais: parede (11), revisita (18).

O agente mantém desempenho robusto, atingindo o objetivo em todos os cenários testados. A mediana igual à rota ótima (8 passos) demonstra que, na maior parte dos casos, o agente encontra trajetórias bastante eficientes.



Fig. 4. Ambiente GridWorld estático

V. LIMITAÇÕES DO MODELO

Embora a configuração rapidez-rapidez tenha apresentado desempenho consistente no ambiente estudado, algumas limitações devem ser reconhecidas. A arquitetura A2C-LSTM depende de escolhas específicas de hiperparâmetros e de uma representação de estado compacta, que podem não capturar todas as variações possíveis da dinâmica do ambiente. Além disso, os testes realizados cobrem um conjunto selecionado de cenários, deixando espaço para avaliações mais amplas em condições futuras.

VI. CONCLUSÃO

Este trabalho apresentou o desenvolvimento, treinamento e avaliação de um agente de navegação baseado no algoritmo *Advantage Actor-Critic* com memória LSTM (A2C-LSTM) em um ambiente *GridWorld* dinâmico e parcialmente observável. O agente foi submetido a uma avaliação sistemática envolvendo 12 combinações entre funções de recompensa e estratégias de treinamento, das quais a configuração *rapidez – rapidez* apresentou o melhor desempenho composto.

A combinação entre uma função de recompensa fortemente orientada à eficiência (penalização elevada por atrasos e altos incentivos por progresso rápido), associada a um treinamento com baixa entropia, permitiu ao agente aprender políticas extremamente eficazes, convergindo para trajetórias curtas e altamente estáveis, mesmo em presença de obstáculos com dinâmica aleatória.

Os resultados experimentais demonstraram que o agente é capaz de:

- convergir rapidamente, alcançando políticas estáveis após poucos milhares de episódios;
- atingir 100% de taxa de sucesso sob dinâmica aleatória de obstáculos;
- executar trajetórias curtas, frequentemente iguais ou próximas do caminho ótimo em ambientes estáticos;
- adaptar-se a bloqueios inesperados, realizando reencaminhamento eficiente com baixo custo adicional;
- manter robustez estatística, mesmo em testes prolongados com múltiplas variações do ambiente.
- Após os primeiros 200 episódios, os episódios convergiram majoritariamente para comprimentos entre 8 e 12 passos.

Em síntese, os achados deste estudo validam a eficácia do modelo A2C-LSTM para tarefas de navegação com dinâmica não trivial e fornecem uma base sólida para extensões mais complexas. Os resultados evidenciam que a integração entre memória temporal, recompensas densas e mecanismos de estabilização constitui uma estratégia poderosa para agentes que operam em ambientes parcialmente observáveis e sujeitos a incertezas estruturais.

VII. REFERÊNCIAS

REFERENCES

- [1] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [2] I. Grondman, L. Buşoniu, G. A. D. Lopes, and R. Babuška, “A survey of actor-critic reinforcement learning: Standard and natural policy gradients,” *IEEE Trans. Systems, Man, and Cybernetics*, vol. 42, no. 6, pp. 1401–1416, 2012.
- [3] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, 2015.
- [4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021.
- [5] J. T. Springenberg et al., “Offline actor-critic reinforcement learning scales to large models,” *arXiv:2402.05546*, 2024.
- [6] M. J. Kochenderfer, T. Wheeler, and K. H. Wray, *Algorithms for Decision Making*, MIT Press, 2023.
- [7] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. 16th Int. Conf. Machine Learning (ICML)*, 1999, pp. 278–287.